# Homework Assignment 2

Brownian Motion in Two Dimensions

EC327 Summer 2026

## Introduction

### Assignment goals

By the end of this assignment you will

- have turned a plain `struct` into a **class** with private data, a constructor, and `const`-correct member functions — the central new skill of week 2
- understand **encapsulation** and the interface-vs-implementation distinction in code you wrote yourself
- have **injected a dependency** (the random engine) instead of hiding it in global state — and seen why that makes your simulation reproducible and testable
- have used the **`<algorithm>` / `<numeric>` library** to compute summary statistics, instead of the hand-rolled loops you wrote in HW1 — the "library check" reflex from Lecture 7
- have stored objects in a `std::vector` and processed them with **range-based `for`**, **structured bindings**, and **lambdas**
- have **watched diffusion emerge from your own data**: a cloud of random walkers spreads out from the origin, and the typical distance grows like $\sqrt{t}$ — this is **Brownian motion**
- have discovered that the histogram of *distance from the origin* is **not** a bell curve — it peaks *away* from zero — and be able to say why

The conceptual goal builds directly on HW1. There, the histogram of a 1D walker's final position was a Gaussian centered at zero — the Central Limit Theorem. Here you go to two dimensions and find two things at once: each *coordinate* is still Gaussian (the CLT again), but the *distance from the origin* follows a different, lopsided shape. Seeing both at once, in data you generated, is the point.

### Working style

**Individual assignment.** Talk with classmates about ideas and approaches; write your own code.

AI use is expected and encouraged, per the course's AI-in-the-learning-loop policy. You must disclose your AI use and you must be able to explain every line of your submission. See the AI disclosure section below.

### Due date

**Thursday, June 4, 2026, 11:59 PM** (end of L11).

### Assignment value

Graded on a 100-point internal scale. HW2 contributes **6.25 course points** (one of four HWs in the 25% HW component).

### Late policy

**30% flat penalty** for any submission up to 1 week late. No credit after that. Documented legitimate excuses (illness, family emergency, etc.) waive the penalty — email the instructor.

### Submission

Submit via the course web app at `curl.bu.edu/ec327`. Submit all required source files plus your reflection and AI disclosure, as a single `.zip`.

### How this assignment is graded — and the bigger picture

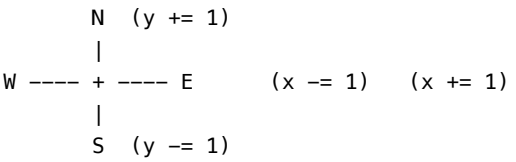
HW1 was the *most* scaffolded assignment of the term: every function signature given, every pitfall named, a step-by-step TODO inside each starter file. **HW2 dials that back one notch.** You still get a working skeleton — the class and function signatures are written for you, and the header files document exactly what each function must do — but the `.cpp` bodies are empty, the parts are described more loosely, and you are expected to make more of the design decisions yourself.

The framing, from the syllabus: every HW is a **product ship**, and different customers hand you different levels of detail. HW1 was the *internal-customer-with-tight-contract* job. **HW2 is the *external customer with a clear requirements doc*** — you get a solid spec and a clean interface to build against, but the implementation, the verification, and the judgment calls are yours. This is on purpose. The scaffolding tapers across the term because *recognizing how much spec you have, and working well at every level of it, is the skill.*

---

## Background — what you are building

In HW1 a walker lived on a number line and stepped $+1$ or $-1$. Now the walker lives on a **2D grid**. At each tick of the clock it picks one of four directions — North, South, East, West — uniformly at random, and takes one unit step that way:



Start a walker at the origin $(0, 0)$ and let it take $t$ steps. Where does it end up? As in HW1, the interesting question is not where *one* walker lands but how a *cloud* of many independent walkers spreads.

Three facts will fall out of your data:

1. **The spread grows like $\sqrt{t}$.** The mean-squared distance from the origin, $\langle r^2 \rangle = \langle x^2 + y^2 \rangle$, equals **exactly** $t$ for this walk. So the typical (root-mean-square) distance is $\sqrt{t}$ — quadruple the steps, and the cloud only doubles in radius. This is the **diffusion law**, the same $\sqrt{N}$ scaling you met in HW1, now in physical distance. It is why a drop of ink in still water spreads slowly, and why it is called *Brownian motion*.
2. **Each coordinate is Gaussian.** The histogram of just the $x$-coordinates of the final positions is a bell curve centered at zero — the Central Limit Theorem again, because $x$ is a sum of many small independent steps. (Its variance is about $t/2$, since on average half the steps move in $x$.)
3. **The distance from the origin is *not* Gaussian.** Histogram the distances $r = \sqrt{x^2 + y^2}$ and you get a lopsided shape that **starts at zero, rises to a peak away from the origin, then tails off**. The single most likely distance is *not* zero. This surprises people. The reason is geometric: there is only one way to be *at* the origin, but there are many grid points at distance, say, 10 — a whole ring of them. (This shape is called a Rayleigh distribution; you do not need that name to do the assignment, only to notice the shape and explain the intuition.)

The point of this homework is to build the 2D simulation, watch these three properties emerge, and have the right reaction to the third one: *"wait — why isn't the most likely distance zero?"*

---

# Setup

You already did the hard environment work in Lab 0 and HW1. **Do not throw your HW1 code away** — your HW1 `Walker` struct and your `histogram` / `print_histogram` functions are the seeds of this assignment. You will be promoting the struct to a class and reusing the histogram almost verbatim.

You will receive a starter directory:

```
hw2_brownian_motion/
├── CMakeLists.txt         # builds the whole thing — extend it if you add files
├── Walker2D.h         # the class declaration + free-function signatures (read this first)
├── Walker2D.cpp           # empty function bodies for you to fill in
├── Histogram2D.h        # histogram signatures (one int version, one double version)
├── Histogram2D.cpp        # empty bodies
├── hw2_main.cpp           # orchestrates the experiment — build it up part by part
├── smoke_test.py          # run this before you submit
├── ec327_zip.py           # builds + checks your submission zip
└── .clang-format          # the course style file
```

Build and run with:

```
cmake -B build
cmake --build build
./build/hw2_main
```

The starter compiles and runs as-is — it just does not do anything useful yet.

---

## Part 1 — `Walker2D`, the class (20 points)

**Goal:** promote the HW1 `struct` to a real class with private data and member functions.

The header gives you this interface (read `Walker2D.h` for the full doc comments):

```
class Walker2D {
 public:
  Walker2D();  // starts at the origin (0, 0)

  void step(std::mt19937& rng);  // one random N/S/E/W unit step

  int x() const;
  int y() const;
  long long r_squared() const;          // x*x + y*y
  double distance_from_origin() const;  // sqrt(r_squared)

 private:
  int x_;
  int y_;
};
```

Two things are deliberately different from HW1:

- **The data is `private`.** Callers reach the position through `x()` / `y()`, never by poking at the members. The accessors are `const` because asking a walker where it is does not move it. This is **encapsulation** — the class controls its own invariants.
- **`step` takes the random engine by reference** instead of hiding a `static` engine inside itself like HW1 did. This is **dependency injection**, and it matters: whoever owns the engine controls the randomness. Seed the engine the same way twice and you get the *same* walk twice — which is exactly what makes a random simulation reproducible and testable. (More on this in Part 5.)

**Implement** the constructor, `step`, and the four accessors in `Walker2D.cpp`. For `step`, draw a number in $\{0, 1, 2, 3\}$ from the injected engine and map it to N/S/E/W.

**In `hw2_main.cpp`**, create one engine, create one `Walker2D`, step it 10 times, and after each step print its position and distance:

```
Walker at (0, 0), distance 0.00
After step 1: (1, 0), distance 1.00
After step 2: (1, 1), distance 1.41
After step 3: (0, 1), distance 1.00
...
```

```
After step 10: (2, -2), distance 2.83
```

**New C++ vocabulary:** `class` vs `struct` · `public:` / `private:` · constructor · `const` member functions · accessor (getter) methods · dependency injection (passing `std::mt19937&` in)

---

## Part 2 — N steps, and the first hint of $\sqrt{t}$ (15 points)

**Goal:** parameterize the number of steps and notice how the distance grows.

A free function is declared for you:

```
void walk_for(Walker2D& w, int N, std::mt19937& rng);  // step N times
```

**In `hw2_main.cpp`**, run one fresh walker for each of $N = 10, 100, 1000$ steps and print its final position and distance. **Run the program a few times** — the answers change every run, and a single walker is noisy.

```
N=10:    final (2, -2),   distance 2.83
N=100:   final (-6, 8),   distance 10.00
N=1000:  final (12, -28), distance 30.46
```

You will notice the distances are *roughly* $\sqrt{10} \approx 3$, $\sqrt{100} = 10$, $\sqrt{1000} \approx 32$ — but only roughly, because one walker tells you almost nothing. Part 3 fixes that by averaging over many.

**New C++ vocabulary:** threading an injected `rng` through a call chain · the intuition that "one trial is noise, the average is signal"

---

## Part 3 — K walkers, mean-squared displacement, *with the standard library* (25 points)

**Goal:** run many walkers and compute summary statistics — this time using the STL, not hand-rolled loops.

A free function is declared for you:

```
std::vector<Walker2D> run_trials(int K, int N, std::mt19937& rng);
```

It runs $K$ independent walkers, each for $N$ steps from the origin, and returns the vector of $K$ finished walkers.

Then, **in `hw2_main.cpp`**, compute three numbers over that vector:

- the **mean distance** from the origin,
- the **mean-squared displacement** $\langle r^2 \rangle = \frac{1}{K} \sum_i r_i^2$,
- the **maximum distance** any walker reached.

**Use the library this time.** In HW1 you wrote the mean as a hand-rolled accumulator loop, and the spec explicitly *forbade* `std::accumulate` so you would understand what the loop does. You have now had Lecture 7. **The ban is lifted.** Compute these with `std::accumulate` (from `<numeric>`) and `std::max_element` (from `<algorithm>`), using lambdas to pull `r_squared()` / `distance_from_origin()` out of each walker. Writing `std::accumulate(v.begin(), v.end(), 0.0, [](double acc, const Walker2D& w){ return acc + w.distance_from_origin(); })` *is* the skill here — naming the loop and reaching for the library version. Hand-rolled loops will not earn full credit on this part.

Run with $K = 10, 100, 1000$, all at $N = 100$ steps, and print:

```
K=10,   N=100:  mean dist = 10.36   <r^2> = 118.4   max dist = 15.81
K=100,  N=100:  mean dist = 8.80    <r^2> = 98.4    max dist = 25.06
K=1000, N=100:  mean dist = 8.97    <r^2> = 101.9   max dist = 32.56
```

What do you notice as $K$ grows? (Hint: with $N = 100$, the *true* mean-squared displacement is exactly 100. Your estimate of $\langle r^2 \rangle$ should close in on it.)

**New C++ vocabulary:** `std::vector<Walker2D>` (a vector of *your own objects*) · `std::accumulate` with a lambda accumulator · `std::max_element` with a comparator lambda · `const Walker2D&` lambda parameters · the "library check" reflex

---

## Part 4 — Two histograms: the bell curve *and* the surprise (30 points)

**Goal:** see both shapes — the Gaussian marginal and the lopsided radial distribution — with your own eyes.

`Histogram2D.h` declares two binning functions and a printer:

```cpp
// Bins integer values (e.g. x-coordinates, which can be negative).
std::map<int, int> histogram_int(const std::vector<int>& values,
                                 int bucket_width);

// Bins non-negative real values (e.g. distances).
std::map<int, int> histogram_distance(const std::vector<double>& values,
                                      double bucket_width);

void print_histogram(const std::map<int, int>& h);
```

`histogram_int` is **your HW1 `histogram` function, essentially unchanged** — including the negative-bucket gotcha (a walker's $x$ can be negative, so `(-3) / 5` rounding toward zero will bite you again; reuse your HW1 `bucket_of` helper). `histogram_distance` is *easier* than HW1 — distances are never negative, so there is no toward-zero rounding trap. For `print_histogram`, do the thing **many of you already figured out in HW1**: scale the bars. The HW1 handout literally said "one $*$ per count," and at $K = 10000$ the peak bucket holds thousands of walkers — so a faithful printer dumps thousands of characters on one row, and most of you quietly rescaled it to fit your terminal. We make that official here:

scale so the **tallest bar is a fixed width** (about 56 $*$). Find the largest count first, then print each bar's length as a fraction of that. (This is a real habit, not a hack — every plotting library normalizes the same way.)

**In `hw2_main.cpp`**, run $K = 10000$ walkers for $N = 100$ steps, then print **two** histograms from the same batch:

1. **The $x$-coordinate histogram** (`histogram_int` on the walkers' `x()` values, `bucket_width = 5`). You should get a **bell curve centered at zero** — the same CLT shape as HW1, because $x$ is a sum of independent steps. Its spread is narrower than HW1's at the same $N$, because only about half the steps move in $x$.
2. **The distance histogram** (`histogram_distance` on the walkers' `distance_from_origin()` values, `bucket_width = 2.0`). You should get a **lopsided hump that starts at zero, rises to a peak away from the origin, and tails off** — *not* a bell curve, and *not* peaked at zero.

```
x-coordinate (K=10000, N=100, bucket=5):
 -25:   *
 -20:   ***
 -15:   ***********
 -10:   ********************************
  -5:   ************************************************************
   0:   ************************************************************
   5:   ************************************
  10:   ***************
  15:   ****
  20:   *

distance from origin (K=10000, N=100, bucket=2):
   0:   ***********
   2:   ******************************
   4:   ************************************************************
   6:   ************************************************
   8:   ************************************************
  10:   ************************************************
  12:   ******************************
  14:   ***********************
  16:   ************
  18:   *******
  20:   ****
  22:   **

<r^2> = 100.38, RMS distance = 10.02
```

(The bars are scaled so the tallest is 56 $*$. Notice the distance histogram's peak sits out around distance 4–10 — **not** at zero — while the $x$-histogram peaks dead centre at zero. That contrast is the whole point of Part 4.)

**In your `output.txt`**, capture both histograms for at least three different $N$ values — for example $N = 100$, $N = 400$, $N = 1600$. Notice that the RMS distance **doubles** when $N$

**quadruples** (the $\sqrt{t}$ law).

**Write 2–3 sentences of observation** in your output file: what shape is each histogram, and — the key question — **why is the most likely distance not zero**, even though the most likely $x$ (and the most likely $y$) *is* zero?

**New C++ vocabulary:** `const std::vector<double>&` parameters · `std::map<int, int>` for sparse counts · reusing your own HW1 code as a library · bucketing reals vs bucketing signed ints

---

## Part 5 — Multi-file build + reflection (10 points)

**Goal:** keep the project in good engineering shape, and articulate what you observed.

Make sure:

- every `.h` has `#pragma once` and declares (never defines) its functions
- the class's data stays `private`; nothing pokes at `x_` / `y_` from outside
- every `.cpp` `#includes` its own header
- `hw2_main.cpp` includes both headers
- `CMakeLists.txt` builds everything — extend it if you added files
- the whole thing compiles cleanly under `-Wall -Wextra` and is `clang-format`-clean

**Write `reflection.md`** — about 100 words (±20) addressing:

- The two histograms: which one matched your HW1 intuition, and which one surprised you? **Why is the radial distribution peaked away from zero?**
- How did the RMS distance scale with $N$? Did quadrupling $N$ double the radius?
- **The engineering question:** `step` takes the random engine as a parameter instead of owning a hidden `static` one. What does that buy you? (Think about what happens if you seed the engine with a fixed number versus `std::random_device`.)

This is the conceptual deliverable. The point of the whole homework lives here.

---

## Required submission

Submit **a single `.zip` file**, uploaded at `curl.bu.edu/ec327`, containing:

| File | What it is |
|---|---|
| `Walker2D.h` `Walker2D.cpp` | The walker class + walk_for / run_trials |
| `Histogram2D.h` `Histogram2D.cpp` | The two histogram binners + the printer |
| `hw2_main.cpp` | The main experiment driver |
| `CMakeLists.txt` | Build configuration (may be the unmodified starter) |
| `output.txt` | Captured output: both histograms for at least 3 N values + your observation |
| `reflection.md` | Your ~100-word reflection |

| File | What it is |
|---|---|
| `ai_disclosure.md` | Your AI use disclosure (template below) |

Do **not** submit the `build/` directory or any compiled binaries. Upload **one `.zip`** — loose files and `.tar.gz` are rejected by the uploader.

A helper, **`ec327_zip.py`**, ships in your starter package. From your project directory:

```
python3 ec327_zip.py build hw2       # writes hw2_submission.zip + checks it
python3 ec327_zip.py check hw2       # re-check an existing zip
```

It bundles the required source files (and picks up `output.txt`, `reflection.md`, `ai_disclosure.md` if present), skips the `build/` directory, and prints a present/missing checklist. You can also zip by hand — the upload page only requires a single valid `.zip`, and shows the same checklist against the files *inside* it after you upload.

---

## Testing your work — the smoke test

Your starter package includes `smoke_test.py`. Run it before submitting:

```
python3 smoke_test.py
```

It checks that the required source files exist, that `cmake -B build && cmake --build build` succeeds, that the binary runs and exits cleanly, that the reported mean-squared displacement is statistically plausible (near $N$ for the largest run), and that **two** histograms are present in the output.

**Passing the smoke test is necessary, not sufficient.** The instructor will read your code, check that your class actually encapsulates its data, check the negative-bucket edge case in `histogram_int`, confirm you used the STL in Part 3 rather than hand-rolled loops, read your reflection, and run additional $(K, N)$ cases during grading.

What the smoke test deliberately does **not** check: code style (we run `clang-format` separately), whether Part 3 actually uses the library, the correctness of negative-`x` bucketing, whether your reflection answers the "why not zero" question, or whether your AI disclosure is honest. Those are instructor-grading territory.

---

## Grading rubric

| Part | Pts | Earns full credit | Common deductions |
|---|---|---|---|
| 1 | 20 | `Walker2D` is a real class: data is `private`, accessors are `const`, constructor starts at the origin, `step` takes the injected `rng` and picks one of four directions correctly | data left `public` (it's still a struct in disguise) · accessors not `const` · `step` ignores the injected engine and uses its own `static` one (defeats the point) · only 2 directions / diagonal moves |
| 2 | 15 | `walk_for` steps exactly $N$ times threading the `rng` through; three runs at $N = 10, 100, 1000$ give plausible distances | off-by-one in the step count · reuses one walker across runs without resetting · drops the `rng` and reseeds |
| 3 | 25 | `run_trials` returns $K$ walkers; mean distance, $\langle r^2 \rangle$, and max distance computed **with `std::accumulate` / `std::max_element` + lambdas**; $\langle r^2 \rangle$ closes on $N$ as $K$ grows | hand-rolled loops instead of the STL (this part is *about* the library) · `r_squared` overflows on large $N$ (use `long long`) · computes mean of $r^2$ as `int` |
| 4 | 30 | Both histograms printed; $x$-histogram is a bell curve centered at 0 (negative buckets correct); distance histogram is the lopsided shape peaked away from 0; observation explains *why* | only one histogram · negative-`x` bucketing wrong (the HW1 gotcha) · distance histogram peaked at 0 (binning bug) · no explanation of the radial peak |

| Part | Pts | Earns full credit | Common deductions |
|---|---|---|---|
| 5 | 10 | Clean header/impl split; data stays private; `reflection.md` is 80–120 words and answers all three prompts including the dependency-injection question; AI disclosure complete | everything dumped in `hw2_main.cpp` · members made public to "make it easier" · reflection skips the *why-not-zero* or the injection question · no AI disclosure |

**Style baseline** (all parts): compiles under `-Wall -Wextra` with no warnings; `clang-format`-clean with the course file. `clang-tidy` is Lab 2 territory — but note that `clang-tidy` would already flag `using namespace std;`, which does not appear in course-distributed code (see L14). Don't write it.

---

## Hints and pitfalls

**Mapping a random draw to a direction.** Draw an `int` in $\{0, 1, 2, 3\}$ from the injected engine and `switch` on it:

```
void Walker2D::step(std::mt19937& rng) {
  std::uniform_int_distribution<int> dir(0, 3);
  switch (dir(rng)) {
    case 0: y_ += 1; break;  // N
    case 1: y_ -= 1; break;  // S
    case 2: x_ += 1; break;  // E
    case 3: x_ -= 1; break;  // W
  }
}
```

The distribution object is cheap to make; you may keep it local. The *engine* is the expensive, stateful thing — and it lives in `main`, passed in by reference, so it advances correctly across every call.

**One engine, seeded once.** Create exactly **one** `std::mt19937` in `main` and thread it through everything:

```
std::mt19937 rng(std::random_device{}());  // random each run
// std::mt19937 rng(42);                   // reproducible — same walk every run
```

If you accidentally create a fresh engine inside `run_trials` or `walk_for`, every walker becomes identical and your histograms collapse to a single spike. (This is the HW1 RNG pitfall wearing a 2D costume.)

**`r_squared` can overflow `int`.** At $N = 1600$ a coordinate can reach the dozens, and $x^2 + y^2$ stays small — but get in the habit now: distances-squared and sums of squares grow fast, so return `long long` from `r_squared()` and accumulate $\langle r^2 \rangle$ as `double`. Integer division (`<r^2>` computed with `int` math) is the other classic way to get a wrong average.

**Negative-x bucketing is the returning HW1 gotcha.** Your $x$-coordinates are signed, so `histogram_int` needs the same `bucket_of` care as HW1: `(-3) / 5 == 0` in C++, but $-3$ belongs in bucket $-5$. Reuse your HW1 helper:

```
int bucket_of(int x, int width) {
  if (x >= 0) return (x / width) * width;
  return -(((-x - 1) / width) + 1) * width;
}
```

`histogram_distance` has no such trap — distances are $\geq 0$, so `static_cast<int>(d / width) * width` (with `width` as a `double` in the division) is enough. Watch the cast: you want to bin the *value*, then label the bucket by its lower edge.

**The library-check reflex (Part 3).** The HW1 spec told you *not* to use `std::accumulate`. That ban was a one-time teaching device. From now on, when you find yourself writing a loop that sums, counts, finds a max, or transforms a range — **stop and ask whether the library already has it.** It almost always does. Lecture 7 was about exactly this: every well-shaped loop is an STL algorithm in disguise.

---

# AI disclosure (required)

Submit `ai_disclosure.md` with the following template filled out. ~100 words total.

```
## AI use disclosure for HW2

### Roles used
(check all that apply)

[ ] Pair programmer — AI drafted or completed code with me
[ ] Coach — AI explained concepts to me
[ ] IT aide — AI helped me with environment / compiler / build issues
[ ] Code reviewer — AI critiqued code I had written
[ ] Test-case generator — AI suggested edge cases
[ ] Spec clarifier — AI helped me unpack ambiguities in the assignment
[ ] Rubber duck — I explained the problem to AI and figured it out mid-explanation
[ ] Cross-language translator — AI showed me C++ equivalents of Python idioms

### What I accepted, what I rejected
(2-3 sentences. Where did AI help? Where did it suggest something wrong or
unhelpful, and how did you catch it?)

### Confidence
(1 sentence. Can you walk through every line of your submission and explain
```

`what it does and why?)`

The disclosure is the practice of the AI policy, not a hurdle. There is no penalty for *using* AI heavily; there is a penalty for using it without disclosing, or for submitting code you cannot explain. See the course AI policy for the full statement.

---

## What comes next

You now have a working Monte Carlo simulation engine and a feel for how random processes spread. **HW3** turns that engine outward: you will pick a problem from a menu and use random sampling to *compute* something — estimating $\pi$, integrating a nasty function, or pricing a simple bet — the Monte Carlo *integration* idea. The spec gets terser again; you will design more of the structure yourself. Keep your `Walker2D` and your histogram code — the habits transfer even when the specific class does not.